

INFORME TÉCNICO I

Investigación y propuesta técnica inicial

Sistema IoT multisensorial para buses eléctricos universitarios

Institución: Escuela Superior Politécnica del Litoral (ESPOL)
Facultad: [FACULTAD O UNIDAD ACADÉMICA]
Carrera: Ingeniería Mecatrónica
Asignatura: [ASIGNATURA / PROYECTO INTEGRADOR]
Autor: Patricio Vásquez ([MATRÍCULA])
Docente: [DOCENTE / TUTOR]
Fecha: Julio de 2026

Resumen ejecutivo

Este informe plantea, desde la fase inicial del proyecto, cómo diseñar un dispositivo embarcado capaz de observar el ambiente, el movimiento y la ubicación de buses eléctricos universitarios. La propuesta parte de un ESP32 y combina sensores de temperatura y humedad, iluminación, presión, aceleración, velocidad angular, campo magnético, posición y nivel acústico relativo. Los datos se validan en el borde, se respaldan localmente y se transmiten mediante MQTT hacia una plataforma web.

La finalidad no es reemplazar sistemas certificados del vehículo, sino construir un prototipo académico extensible para estudiar condiciones de operación, calidad del entorno, disponibilidad de la flota y comportamiento del trayecto. Se propone una arquitectura modular en PlatformIO, pruebas separadas por componente y un contrato de telemetría versionado. La solución prioriza continuidad: un sensor o una conexión que falle no debe detener el resto del dispositivo ni producir valores ficticios.

Resultado de la investigación

La alternativa recomendada para la primera campaña es un perfil WiFi sobre ESP32, MQTT con TLS, microSD y página local de diagnóstico. El SIM800L se conserva como línea experimental, pero no como transporte principal, debido a cobertura 2G, alimentación por picos y compatibilidad TLS. La arquitectura permite migrar después a un módem LTE sin reescribir sensores ni telemetría.

Índice general

Resumen ejecutivo	I
1. Planteamiento del proyecto	1
1.1. Contexto	1
1.2. Pregunta de investigación	1
1.3. Objetivos	1
1.3.1. Objetivo general	1
1.3.2. Objetivos específicos	1
1.4. Alcance inicial y exclusiones	2
2. Fundamentos y utilidad esperada	3
2.1. Cadena de valor del dato	3
2.2. Por qué utilizar MQTT	3
2.3. Qué se espera obtener	3
3. Selección de hardware	5
3.1. Controlador ESP32	5
3.2. Sensores propuestos	5
3.3. Comunicación primaria y alternativa	6
3.4. Mapa de pines propuesto	6
4. Arquitectura propuesta	7
4.1. Capas del firmware	7
4.2. Organización del repositorio	7
4.3. Planificación temporal	8
4.4. Contrato de telemetría	8
5. Plataforma de visualización propuesta	9
5.1. Página local	9

5.2. Plataforma cloud	9
6. Metodología de desarrollo y pruebas	10
6.1. Entornos PlatformIO	10
6.2. Fases	10
6.3. Criterios de aceptación iniciales	10
7. Riesgos y decisiones abiertas	11
8. Conclusiones de la propuesta	12

Índice de figuras

2.1. Cadena propuesta para convertir una medición en telemetría utilizable.	3
4.1. Separación por responsabilidades propuesta para el firmware.	7
5.1. Arquitectura cloud recomendada para separar ingestión, persistencia y presentación.	9

Planteamiento del proyecto

1.1 Contexto

Una operación universitaria con buses eléctricos necesita información que normalmente se encuentra dispersa: posición del vehículo, condiciones ambientales interiores, movimiento, eventos bruscos, conectividad y estado del propio dispositivo. La observación manual sirve para ensayos puntuales, pero no para comparar rutas, horarios o tendencias. Un nodo IoT embarcado permite transformar esas variables físicas en una serie temporal consultable.

El proyecto VehicleSense se formula como un prototipo de instrumentación, no como un sistema de seguridad funcional. Sus resultados pueden apoyar investigación, mantenimiento exploratorio y visualización operacional; cualquier decisión crítica debe contrastarse con instrumentos calibrados y con los sistemas homologados del bus.

1.2 Pregunta de investigación

¿Cómo construir, con componentes accesibles y software modular, un dispositivo que adquiera variables heterogéneas dentro de un bus eléctrico, detecte datos inválidos, mantenga un respaldo local y publique telemetría útil sin bloquearse ante fallos parciales?

1.3 Objetivos

1.3.1 Objetivo general

Diseñar e implementar una arquitectura IoT multisensorial extensible para monitorear buses eléctricos universitarios mediante adquisición local, validación, almacenamiento y comunicación MQTT.

1.3.2 Objetivos específicos

- Seleccionar sensores y buses de comunicación compatibles con el ESP32.
- Definir magnitudes, unidades, intervalos y criterios de validez.
- Separar control, sensores, almacenamiento, red y presentación en módulos.

- Diseñar un payload JSON versionado que omita números inválidos.
- Proponer diagnóstico local, actualización OTA y plataforma cloud.
- Elaborar una metodología reproducible de calibración e integración.

1.4 Alcance inicial y exclusiones

El prototipo medirá variables ambientales y dinámicas con componentes de bajo costo. No accederá al CAN del vehículo, no estimará el estado de carga de la batería y no emitirá alarmas de seguridad certificadas. Tampoco se considera en la primera fase una calibración metrológica trazable. El audio se procesará en características agregadas; no se almacenará voz cruda.

Fundamentos y utilidad esperada

2.1 Cadena de valor del dato

Una lectura aislada no es todavía información. El valor aparece cuando se conoce su unidad, instante, procedencia y calidad. La cadena propuesta es:

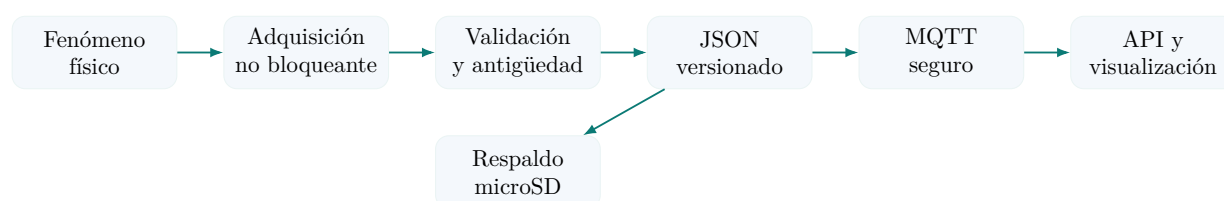


Figura 2.1: Cadena propuesta para convertir una medición en telemetría utilizable.

2.2 Por qué utilizar MQTT

MQTT es un protocolo de publicación/suscripción liviano y apropiado para entornos restringidos [4]. El dispositivo no necesita conocer a todos los consumidores: publica en temas definidos y un broker distribuye los mensajes. Esta separación permite que el dashboard, el almacenamiento histórico y futuras reglas de alerta evolucionen sin cambiar el firmware de adquisición.

Para la telemetría se propone QoS 1 cuando el cliente utilizado lo permita, con identificador de mensaje y confirmación PUBACK. La microSD complementa, pero no sustituye, esta garantía. TLS y credenciales por dispositivo son requisitos del despliegue; MQTT en texto plano se limita al laboratorio.

2.3 Qué se espera obtener

- Series de temperatura, humedad, iluminación y presión dentro del vehículo.
- Trayectorias, velocidad y calidad de fix del GPS.
- Magnitudes de aceleración, giro y campo magnético para explorar eventos.
- Evidencia de continuidad, reinicios, pérdida de red y recuperación.
- Datos exportables para informes, gráficas y análisis posteriores.

La IMU no identifica por sí sola una maniobra peligrosa. Un evento como frenado brusco requiere definir el eje longitudinal, filtrar vibración, estimar el marco del vehículo y validar umbrales con recorridos controlados. Del mismo modo, el nivel del micrófono en dBFS es relativo y no equivale a dB SPL sin calibración acústica.

Selección de hardware

3.1 Controlador ESP32

El ESP32 integra WiFi de 2,4 GHz, múltiples UART, SPI, I2C e I2S, además de recursos de seguridad y memoria adecuados para un nodo IoT [1]. La placa DevKit V1 facilita programación USB y depuración. Sus limitaciones más relevantes son memoria finita, pines de arranque y la necesidad de evitar operaciones largas dentro del loop.

3.2 Sensores propuestos

Módulo	Magnitud	Interfaz	Justificación y cautela
DHT11	Temperatura y humedad	GPIO27, señal digital	Económico y suficiente para prototipo; baja resolución y lectura máxima cada 2 s.
NEO-6M	Posición, velocidad, altitud, satélites	UART2, GPIO32/33	NMEA ampliamente soportado; necesita cielo visible y puede tardar en fijar posición [2].
ADXL345	Aceleración triaxial	I2C, dirección típica 0x53	Permite medir gravedad y movimiento; requiere calibración de offset y escala [3].
L3G4200D	Velocidad angular triaxial	I2C, 0x69/0x68	Complementa al acelerómetro; deriva y sesgo impiden obtener orientación absoluta sin fusión.
HMC5883L	Campo magnético	I2C, 0x1E	Útil para rumbo experimental; sensible a metal, corriente y clones QMC5883L.
BMP180	Presión y temperatura interna	I2C, 0x77	Permite estimar altitud; la presión de referencia cambia con el clima.

Módulo	Magnitud	Interfaz	Justificación y cautela
BH1750	Iluminancia	I2C, 0x23/0x5C	Entrega lux directamente; la geometría, sombra y saturación afectan la interpretación.
INMP441	Señal acústica digital	I2S	Evita entrada analógica; se propone extraer características y descartar audio crudo.
microSD	Respaldo local	SPI, GPIO18/19/23/5	Conserva muestras durante cortes; requiere FAT y manejo de extracción/energía.

El ADXL345 admite rangos seleccionables y salida digital por I2C/SPI [3]. El NEO-6 ofrece UART y otros buses, pero el breakout debe verificarse: la tensión de entrada de la placa no siempre coincide con la del chip [2].

3.3 Comunicación primaria y alternativa

El perfil recomendado utiliza WiFi porque simplifica TLS, OTA y pruebas en campus. El SIM800L se evalúa como experimento de conectividad GPRS, no como requisito de la primera versión. Sus riesgos son cobertura 2G, picos de corriente cercanos a 2 A, latencia de comandos AT y soporte TLS dependiente del firmware. La interfaz de red se diseña para aceptar después un módem LTE como SIM7600.

3.4 Mapa de pines propuesto

Cuadro 3.2: Asignación de pines para el prototipo extendido.

Bloque	Señal	GPIO	Observación
DHT11	DATA	27	Pull-up de 4,7–10 k Ω si el módulo no lo incluye.
GPS	RX2 / TX2	32 / 33	GPS TX hacia GPIO32; 9600 baud.
I2C	SDA / SCL	21 / 22	Bus compartido por GY-801 y BH1750.
microSD	SCK/MISO/MOSI/CS	18/19/23/5	CS alto durante arranque; GPIO5 es strapping.
INMP441	SCK/WS/SD	Según configuración	Bus I2S independiente; documentar cableado final.
SIM800L experimental	RX1 / TX1	16 / 17	Fuente externa estable y tierra común.

Arquitectura propuesta

4.1 Capas del firmware

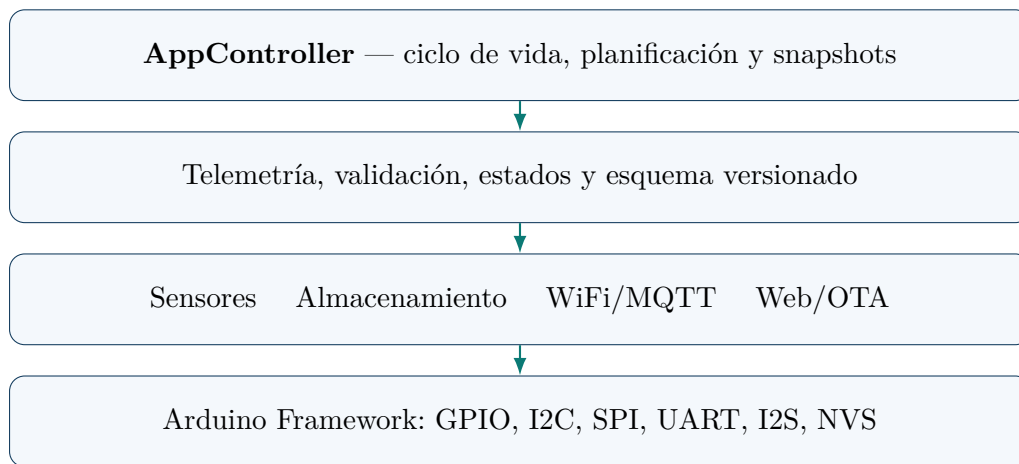


Figura 4.1: Separación por responsabilidades propuesta para el firmware.

Cada sensor expone un contrato equivalente a `begin()`, `update(nowMs)`, `isValid()` y `getData()`. No conoce JSON, MQTT ni la interfaz web. La telemetría trabaja con datos tipados y el transporte recibe un documento ya validado. Así, cambiar de WiFi a LTE no afecta el driver del acelerómetro.

4.2 Organización del repositorio

<code>include/</code>	config, pines y secretos de ejemplo
<code>src/app/</code>	controlador principal
<code>src/sensors/</code>	DHT11, GPS, BH1750, audio y GY-801
<code>src/storage/</code>	microSD y cola offline
<code>src/communication/</code>	WiFi, MQTT y modem celular
<code>src/telemetry/</code>	datos, validacion y JSON
<code>src/web/</code>	dashboard local, API y OTA
<code>src/diagnostics/</code>	firmware de prueba por environment
<code>test/</code>	pruebas Unity nativas
<code>docs/</code>	cableado, arquitectura y procedimientos

4.3 Planificación temporal

El GPS se procesa en cada iteración para no perder caracteres. IMU y audio pueden requerir intervalos cortos; DHT11 no debe leerse más rápido que 2 s. La telemetría se publica a un intervalo superior, por ejemplo 10 s, con una instantánea coherente. Las reconexiones usan backoff y `millis()`, sin `delay()` prolongado.

4.4 Contrato de telemetría

El payload incluirá versión de esquema, identificadores, tiempo, valores y banderas de validez. Los datos NaN, infinitos, obsoletos o asociados con una bandera falsa se omiten. Un ejemplo reducido es:

```
{
  "schema_version": 3,
  "device_id": "esp32-devkit-01",
  "vehicle_id": "bus-prototipo-01",
  "uptime_ms": 123456,
  "temperature_c": 27.4,
  "dht_valid": true,
  "gps_valid": false,
  "accel_magnitude_mps2": 9.79,
  "sd_valid": true,
  "mqtt_connected": true
}
```

No se debe inventar tiempo Unix: hasta sincronizar NTP o GPS se informa uptime y fuente temporal. Tampoco se asigna cero a una lectura ausente, porque cero podría ser un valor físicamente válido.

Plataforma de visualización propuesta

5.1 Página local

El ESP32 ofrecerá un Access Point y una página sin dependencias externas. Su propósito es diagnóstico a corta distancia: lecturas básicas, estado de microSD, WiFi, MQTT, uptime y actualización OTA. La sección administrativa utilizará autenticación adicional. La primera carga seguirá realizándose por USB.

La web local no reemplaza la plataforma central. Mantiene poca historia y su acceso depende de proximidad al vehículo. Como criterio de privacidad, la propuesta inicial evita mostrar un mapa local; la posición se reserva al sistema cloud autenticado.

5.2 Plataforma cloud

Se propone un broker MQTT, un consumidor backend persistente, PostgreSQL y una SPA. El frontend consultará API REST para historia y WebSocket para actualización en vivo. Las pantallas previstas son dashboard, mapa, vehículos, alertas, viajes, analítica, reportes, dispositivos y configuración.

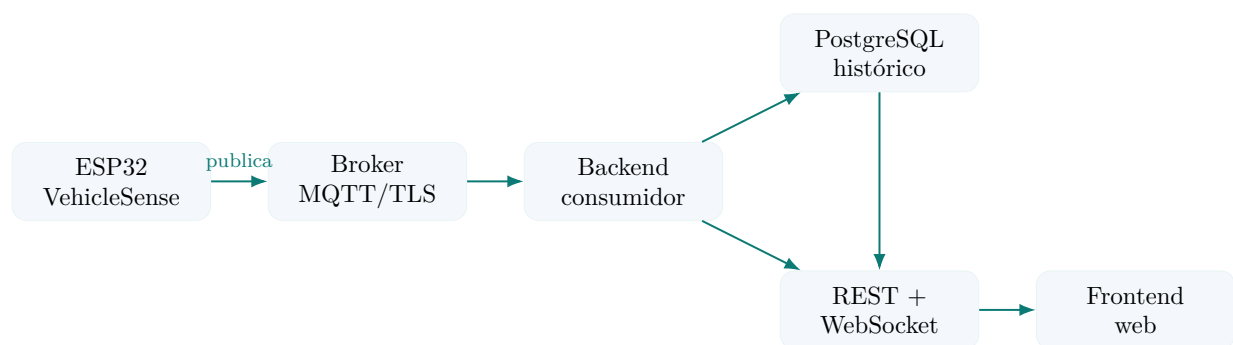


Figura 5.1: Arquitectura cloud recomendada para separar ingestión, persistencia y presentación.

Metodología de desarrollo y pruebas

6.1 Entornos PlatformIO

La estrategia es compilar firmware especializado que no dependa de todo el hardware. Se prevén entornos para DHT11, GPS, scanner I2C, cada chip del GY-801, BH1750, microSD, WiFi, MQTT, audio, web local, OTA e integración completa. Las pruebas de serialización y validación se ejecutan también como Unity nativo.

6.2 Fases

1. **Inspección eléctrica:** tensiones, tierras, fuente y continuidad.
2. **Detección:** Serial, UART, scanner I2C y montaje SPI.
3. **Lectura individual:** rango, finitud, respuesta al estímulo y fallo.
4. **Calibración:** acelerómetro, barómetro y criterios para magnetómetro.
5. **Integración local:** payload, microSD, AP, API y OTA.
6. **Comunicación:** TLS, ACL, QoS, reconexión y cola offline.
7. **Ensayo vehicular:** recorrido controlado, repetibilidad y comparación.

6.3 Criterios de aceptación iniciales

- Todos los entornos compilan y el `main.cpp` conserva delegación mínima.
- La desconexión de un sensor no reinicia el equipo.
- El JSON nunca contiene NaN ni coordenadas ficticias.
- El respaldo continúa cuando MQTT no está disponible.
- La red y el broker se recuperan automáticamente.
- Credenciales reales quedan fuera del repositorio.
- Las pantallas distinguen datos reales, simulados y obsoletos.

Riesgos y decisiones abiertas

Riesgo	Impacto	Tratamiento propuesto
Fuente y ruido eléctrico del bus	Reinicios, lecturas I2C erráticas o daño	Regulación automotriz, protección, filtrado y pruebas antes de instalación.
Clones del GY-801	Dirección o driver incompatible	Scanner más verificación de registros de identidad; validez por subchip.
GPS bajo techo	Flujo NMEA sin fix	Diferenciar UART activo de posición válida; ensayo exterior.
Altitud BMP180	Error por clima y referencia	Calibración reciente con altitud conocida; registrar presión de referencia.
Perturbación magnética	Rumbo incorrecto	Calibrar dentro del montaje final o excluir rumbo de decisiones.
Pérdida de conectividad	Huecos en la nube	JSONL local y cola acotada con replay.
Seguridad de OTA/MQTT	Acceso o firmware no autorizado	WPA2, Basic Auth local, TLS, ACL, USB de recuperación; firmas en roadmap.
Privacidad GPS/audio	Exposición de ruta o conversaciones	Control de roles, retención limitada y solo características acústicas.

Conclusiones de la propuesta

VehicleSense es viable como plataforma académica si se construye por capas y se documenta la calidad de cada dato. La investigación conduce a una implementación inicial basada en ESP32, sensores digitales, microSD y WiFi/MQTT seguro. Esta opción reduce incertidumbre antes de introducir una red celular.

El valor principal del proyecto no reside en acumular sensores, sino en asegurar que cada medición tenga unidad, validez, contexto temporal y procedimiento de verificación. La siguiente fase debe registrar calibraciones y pruebas físicas sin confundir salidas de ejemplo con evidencia experimental. Después se integrarán almacenamiento, comunicación y visualización de extremo a extremo.

Bibliografía

- [1] Espressif Systems. *ESP32 Series Datasheet, versión 5.2*. https://documentation.espressif.com/esp32_datasheet_en.pdf
- [2] u-blox. *NEO-6 Data Sheet, GPS.G6-HW-09005*. https://content.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet_%28GPS.G6-HW-09005%29.pdf
- [3] Analog Devices. *ADXL345: 3-Axis Digital Accelerometer Data Sheet*. <https://www.analog.com/media/en/technical-documentation/data-sheets/adxl345.pdf>
- [4] OASIS. *MQTT Version 3.1.1, OASIS Standard*. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
- [5] PlatformIO. *Espressif 32 development platform*. <https://docs.platformio.org/en/latest/platforms/espressif32.html>
- [6] ArduinoJson. *Documentation and serialization model*. <https://arduinojson.org/>